PROJECT EXPLAINER

Offline map tiles

What changed, how it behaves, and why it matters

The goal is to let the campus map reuse tiles the user has already loaded when the network is unavailable. It also makes the cache visible, so the user can tell whether offline support is actually active.

01 How the tile cache works

1. Request

Leaflet requests map tiles from the OpenStreetMap tile servers as the map is viewed.

2. Reuse + refresh

Workbox returns a cached tile promptly and refreshes it in the background when online.

3. Offline use

With no connection, previously cached tiles can still be used. New areas are not preloaded.

02 Changes made

- **Caching strategy:** The Workbox rule uses StaleWhileRevalidate for OSM tiles: cached content is available quickly, and online requests update it in the background.
- **Bounded cache:** The stable cache name is osm-map-tiles-cache. It holds at most 1,000 tiles, with a 15-day lifetime. This bounds storage while allowing reuse across app updates.
- **Successful responses only:** Leaflet requests tiles in CORS mode, and Workbox only stores HTTP 200 responses. Failed requests are not treated as valid cached map images.
- **Visible diagnostics:** The map reports online/offline state, whether a service worker controls the page, cached tile count, and tile-load warnings.
- **Test support:** A helper reads Cache Storage without creating an empty cache. Unit tests cover an uncontrolled page, empty cache, cached tile count, and storage errors.

KEY LIMIT Offline coverage is limited to tiles the user has already viewed. This is not a pre-downloaded map of the whole campus.

DEMONSTRATION GUIDE

How to verify it

A short test flow and a professor-friendly summary

03 Verify the behavior

1

Build and preview

From the repository root, run npm run build:client, then npm run preview --workspace=client. Open the local URL printed by the preview command. Service-worker caching is tested in the production preview, not the usual Nuxt dev server.

2

Confirm control

In the map status panel, confirm the service worker controls the page. If it does not, reload after the worker installs. Service workers need HTTPS, except localhost is allowed for local testing.

3

Populate cache

While online, pan or zoom the map. The panel should report a cached tile count. DevTools > Application > Cache Storage should show osm-map-tiles-cache.

4

Go offline

Use DevTools > Network > Offline and revisit an area already viewed. Cached tiles should still render. A never-viewed area can be blank; reconnect and view it to cache it.

04 Why this is important

A PWA configuration can claim offline support without proving that any map tile was saved. The visible diagnostics make setup problems easier to spot; response filtering avoids caching failed tile responses; and the size/age limits manage browser storage. Together, these changes improve resilience and make the feature demonstrable.

PRESENTATION SUMMARY

"We cache map tiles as the user views the map. Online, cached tiles are refreshed in the background; offline, previously viewed areas remain available. The map shows whether caching is active and how many tiles are stored, but it does not promise offline coverage for areas that were never loaded."

Files changed

- client/nuxt.config.ts - Workbox tile strategy, cache limits, and successful-response filter
- client/pages/index.vue - network/cache status and tile failure messages on the map
- client/utils/map-tile-cache.ts - shared cache name and status inspection helper
- client/test/utils/map-tile-cache.spec.ts - unit tests for cache status behavior
- client/test/tsconfig.json - Jest types scoped to test TypeScript files
- DEVELOPERS_GUIDE.md - offline caching behavior and limitations